CHUYÊN ĐỀ · MÔN LẬP TRÌNH ĐA NỀN TẢNG

Quản lý trạng thái với Provider và ChangeNotifier trong Flutter

Ứng dụng thực tế Course Registration & Phòng thí nghiệm Provider Lab

---

SINH VIÊN THỰC HIỆN

Đặng Công Thắng · Lớp 23KTMT2 · MSSV: 106230061

Võ Văn Thái · Lớp 23KTMT1 · MSSV: 106230027

GIẢNG VIÊN HƯỚNG DẪN & ĐƠN VỊ

TS. Nguyễn Duy Nhật Viễn

Khoa Điện tử Viễn thông · Trường Đại học Bách khoa – ĐH Đà Nẵng

Mã nguồn: github.com/cityengin/course-registration

Phân công nhiệm vụ & Tinh thần cộng tác

Đặng Công Thắng — 52%

- **Thiết kế kiến trúc tổng thể:** Định hình luồng dữ liệu 1 chiều, Model `Course`, danh mục 10 học phần ĐHBK (2–4 TC).
- **Cài đặt RegistrationNotifier:** Trung tâm quản lý trạng thái và toàn bộ logic nghiệp vụ (ràng buộc 14–24 tín chỉ, ngăn trùng môn).
- **Xây dựng 2 màn hình nghiệp vụ:** `CourseListScreen` (chính) và `RegisteredScreen` (học phần đã chọn).
- **Tách widget chuyên biệt:** `CourseTile`, `CreditBadge`, `SummaryBar`.
- **Kiểm thử & Báo cáo:** Thiết kế `DemoPanel` trực quan, 7 unit & widget test tự động, slide thuyết trình.

Võ Văn Thái — 48%

- **Xây dựng ProviderLabScreen:** Màn hình phòng thí nghiệm độc lập phục vụ đo lường thực nghiệm số lần build.
- **Cài đặt LabNotifier:** Minh họa Provider ở phạm vi cục bộ (scoped provider), tự động giải phóng tài nguyên với `dispose()`.
- **Bộ công cụ RebuildTracker:** Cài đặt `RebuildTracker` kết hợp chip đếm `BuildCountChip` đếm chính xác số lần hàm `build()` thực thi.
- **Thực nghiệm so sánh:** Đối chiếu số liệu rebuild thực tế giữa `setState` truyền thống và 4 cách đọc State trong Provider.

Tổng quan đề tài & Đặt vấn đề: Tại sao cần Provider?

Vấn đề cốt lõi: Cùng một dữ liệu (danh sách học phần, tổng số tín chỉ) cần hiển thị tại **4 vị trí độc lập**. Nếu chỉ dùng `setState()`: Bị **Prop Drilling** (truyền tham số qua nhiều tầng) và **Over-rebuilding** (vẽ lại toàn bộ cây Widget con không cần thiết).

Course Registration

Ứng dụng đăng ký học phần thực tế

- 10 học phần mẫu ĐHBK (2–4 tín chỉ, tổng 30 TC).
- Ràng buộc nghiệp vụ: tối thiểu 14 TC, tối đa 24 TC.
- Đồng bộ trạng thái 2 chiều tức thì giữa 2 màn hình.

Demo Panel

Bảng điều khiển trực quan hóa xung nhịp

- 4 kịch bản thao tác nhanh (Đủ tối thiểu, Quá tải, Trùng môn, Reset).
- Nhật ký `notifyListeners()` thời gian thực.
- Chứng minh: *Chỉ notify khi state thật sự đổi.*

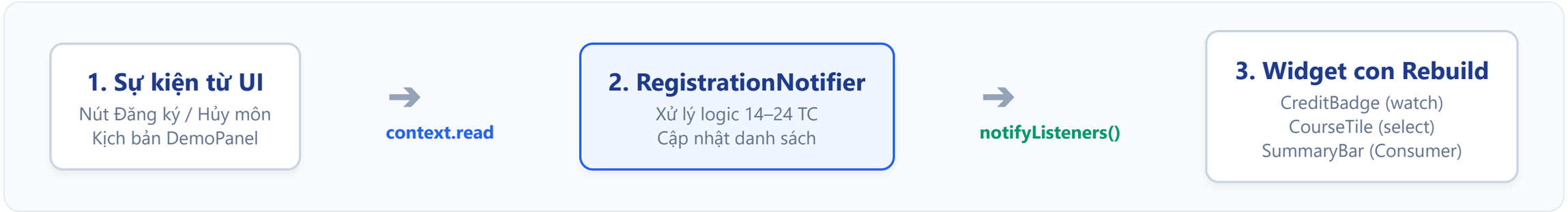
Provider Lab

Phòng thí nghiệm đo lường Rebuild

- Đối đầu trực diện `setState` vs Provider.
- Đo số lần build của 4 cách đọc State.
- Trực quan hóa qua chip `Build: n` đổi màu.

Công nghệ: **Flutter (Material 3)** · Thư viện **provider: ^6.1.5** · **7 bài test tự động** (5 unit test + 2 widget test đạt 100%).

Kiến trúc phân lớp và Luồng dữ liệu một chiều



Tầng Giao diện (UI)

`lib/views/`, `lib/widgets/`

- Chỉ chịu trách nhiệm vẽ giao diện và bắt sự kiện chạm của người dùng.
- Tuyệt đối không chứa logic cộng trừ tín chỉ hay phán đoán hợp lệ.

Tầng Trạng thái (State)

`lib/notifiers/`

- Lưu trữ trạng thái sống, xử lý logic kiểm tra điều kiện.
- Chủ động quyết định chính xác thời điểm nào cần phát `notifyListeners()`.

Tầng Dữ liệu (Data)

`lib/models/`, `lib/data/`

- Model `Course` bất biến (immutable) với các trường dữ liệu cố định.
- Danh mục mẫu gồm 10 môn học độc lập.

RegistrationNotifier: Trái tim logic nghiệp vụ

1. Tính đóng gói (Encapsulation)

Biến `_registered` là **private**. Bên ngoài chỉ có thể đọc thông qua getter `List.unmodifiable(_registered)`, ngăn chặn việc UI tự ý chỉnh sửa danh sách.

2. Trạng thái dẫn xuất (Derived State)

`totalCredits` và `status` được tính tức thì từ danh sách bằng toán tử `fold()`. Đảm bảo nguyên tắc **Single Source of Truth**.

3. Ràng buộc tín chỉ & Notify có điều kiện

Tối thiểu 14 TC, tối đa 24 TC, chống trùng môn. Nếu có lỗi: trả về chuỗi `String?` và **TUYỆT ĐỐI KHÔNG GỌI `notifyListeners()`**, tránh rebuild thừa!

```
// lib/notifiers/registration_notifier.dart (rút gọn) class RegistrationNotifier extends ChangeNotifier { static const int minCredits = 14; static const int maxCredits = 24; final List<Course> _registered = []; List<Course> get registered => List.unmodifiable(_registered); int get totalCredits => _registered.fold( 0, (sum, course) => sum + course.credits); String? register(Course course) { if (isRegistered(course.id)) { return 'Đã đăng ký!'; } if (totalCredits + course.credits > maxCredits) { return 'Vượt quá giới hạn tối đa 24 tín chỉ!'; } _registered.add(course); notifyListeners(); // CHỈ THÔNG BÁO KHI STATE THẬT SỰ ĐỔI! return null; } }
```

Cung cấp State: ChangeNotifierProvider Toàn cục vs Cục bộ


```
// 1. Toàn cục (lib/main.dart): Dùng chung mọi màn hình
ChangeNotifierProvider( create: (_) => RegistrationNotifier(),
child: MaterialApp( home: const CourseListScreen(), ), )
```

```
// 2. Cục bộ (lib/lab/provider_lab_screen.dart): Chỉ trong Lab
ChangeNotifierProvider( create: (_) => LabNotifier(), child: const
_ProviderLabView(), ) // Khi người dùng bấm Back rời khỏi Lab: //
Provider sẽ tự động kích hoạt dispose() dọn sạch RAM!
```

Tiêu chí so sánh	Provider Toàn cục (Global)	Provider Cục bộ (Scoped / Local)
Vị trí khai báo	Bọc ngoài cùng quanh `MaterialApp` (`main.dart`)	Bọc riêng trong màn hình `ProviderLabScreen`
Phạm vi truy cập	Mọi màn hình: `CourseListScreen`, `RegisteredScreen`	Chỉ các widget con nằm bên trong màn hình Lab
Vòng đời (Lifecycle)	Tồn tại suốt thời gian ứng dụng chạy	Tự động gọi `dispose()` giải phóng RAM khi thoát màn hình
Mục đích kiến trúc	Đồng bộ dữ liệu học phần chung trên toàn hệ thống	Phục vụ đo đạc thực nghiệm, cô lập hoàn toàn khỏi dữ liệu thật

Bốn cách đọc State trong Provider

`setState (Cả cây)` → `context.watch (Cả Widget)` → `Consumer (Chỉ hàm builder)` → `context.select (Chỉ khi giá trị đổi)` → `context.read (0 Rebuild)`

Cách đọc State	Hành vi Rebuild	Cơ chế hoạt động	Áp dụng trong dự án
`context.watch<T>()`	Mỗi lần `notifyListeners()`	Đăng ký widget làm subscriber toàn phần	`CreditBadge` (AppBar), `RegisteredScreen`
`<Consumer<T>>`	Chỉ chạy lại hàm `builder`	Khoanh vùng rebuild; có tham số `child` giữ lại UI tĩnh	`SummaryBar` (thanh tiến độ ở chân màn hình)
`context.select<T, R>()`	**CHỈ** khi giá trị con `R` đổi	Lọc điều kiện, bỏ qua các thay đổi khác trong Notifier	`CourseTile` (chỉ theo dõi `isRegistered(id)`)
`context.read<T>()`	**KHÔNG BAO GIỜ**	Lấy tham chiếu Notifier để gọi method, không đăng ký lắng nghe	Các nút bấm `onPressed`: Đăng ký, Hủy môn, Clear

⭐ QUY TẮC VÀNG: `watch` / `select` / `Consumer` đặt trong hàm `build()`; `context.read` đặt trong callback sự kiện (`onPressed`)!